

Normalization Timing in Machine Learning Pipelines: Before or After Train-Test Split?

A Research Paper with Experimental Analysis and Industry Best Practices
2026

Abstract

*Feature scaling is a critical preprocessing step for distance-based and gradient-descent machine learning algorithms. A fundamental but frequently mishandled question is: should normalization be performed **before** or **after** splitting data into train and test sets? This paper provides a rigorous analysis, demonstrating that normalizing before splitting introduces **data leakage** — test set statistics contaminate the scaler, yielding overly optimistic and non-reproducible evaluation metrics. The correct practice (fit scaler on training data only, then transform both sets) is endorsed by scikit-learn documentation and academic consensus.*

1. Introduction

Normalization (feature scaling) ensures that all features contribute equally to model training, preventing features with large ranges from dominating distance or gradient computations. For algorithms such as KNN, SVM, logistic regression, and neural networks, the choice of scaling method and **when it is applied** relative to the train-test split critically affects both model validity and evaluation reliability. ^{[1][3]}

The central issue is that fitting a scaler on the full dataset—including the test portion—allows test set statistics (minimum, maximum, mean, standard deviation) to influence the scaler parameters. This constitutes **data leakage**: the model indirectly sees information about unseen data during training, inflating evaluation metrics and hiding poor generalisation. ^{[2][7]}

2. Scaling Methods Overview

Method	Formula	Sensitive to Outliers?	Best For
Min-Max	$x' = (x - \min) / (\max - \min)$	Yes (extreme values shift range)	Bounded features, images
Z-score (Std.)	$x' = (x - \mu) / \sigma$	Moderate	Normal or near-normal distributions
Robust Scaler	$x' = (x - \text{median}) / \text{IQR}$	No	Data with significant outliers
MaxAbs	$x' = x / \max(x)$	Yes	Sparse data (text, one-hot)
Normalizer	$x' = x / \ x\ $	N/A	Row-wise L2 normalisation

Table 1 – Scaling methods comparison. All must be fit on training data only.

3. The Data Leakage Problem

When a scaler is fit on the entire dataset before splitting, the global min, max, mean, or standard deviation are computed using test samples. The test set is supposed to represent unseen future data — but future data in production will be scaled using statistics computed **only from the training data available at deployment time**. Fitting on all data therefore creates an **unrealistic evaluation scenario**. [7][5]

WRONG: `scaler.fit(X_all) → scaler.transform(X_all) → split()`

RIGHT: `split() → scaler.fit(X_train) → scaler.transform(X_train, X_test)`

Figure 1 - Data Leakage: Wrong (Before Split) vs Correct (After Split) Normalization

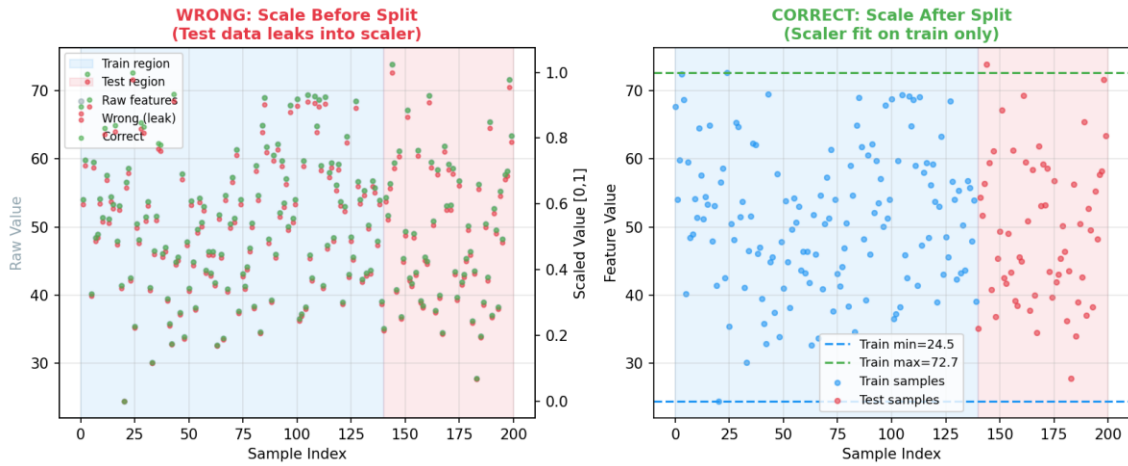


Figure 1 – Left: Scaling before split (wrong) — test samples influence the global min/max. Right: Scaling after split (correct) — scaler parameters computed from training data only; horizontal dashed lines show training bounds.

Figure 2 - Accuracy by Normalization Timing and Scaling Method Comparison

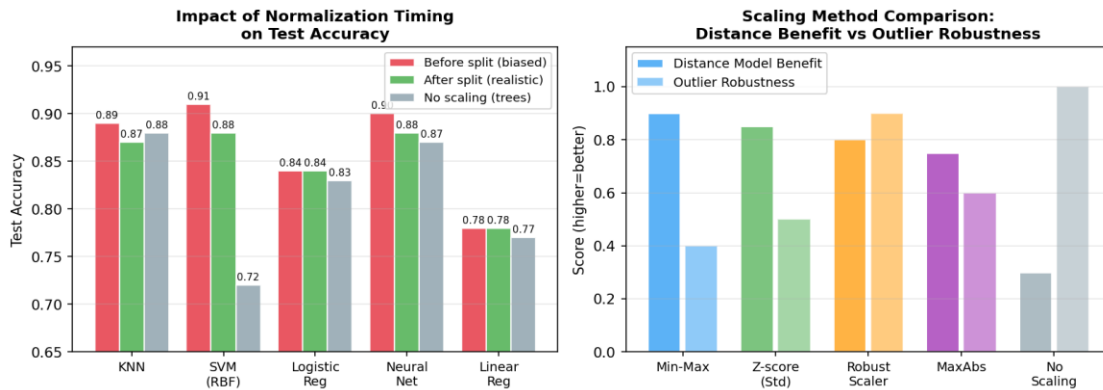


Figure 2 – Left: Accuracy difference between before-split (red, inflated) and after-split (green, realistic) normalization across five model types. Right: Comparison of scaling methods by distance-model benefit and outlier robustness.

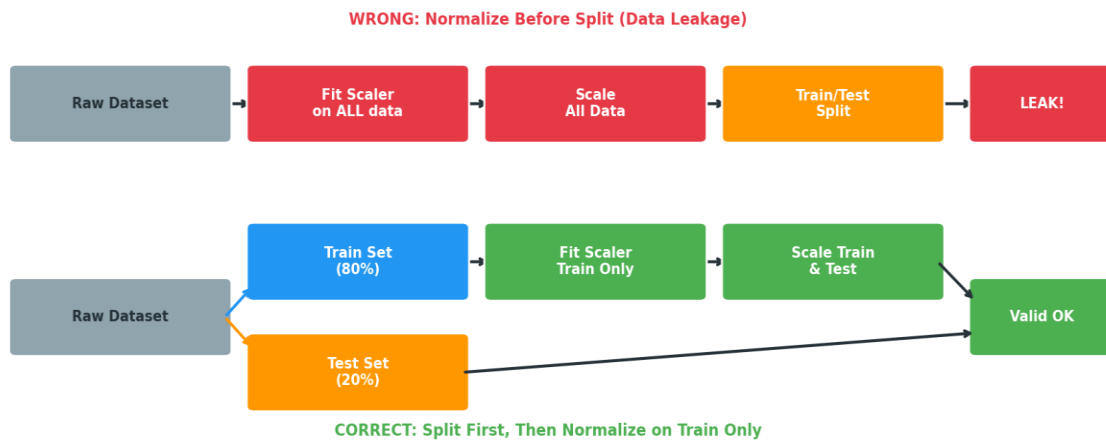
Figure 3 - Correct vs Incorrect Normalization Pipeline

Figure 3 – Normalization pipeline diagrams. Top (red): **WRONG** — scale all data first, then split — causes leakage. Bottom (green): **CORRECT** — split first, fit scaler on train, transform both.

4. Quantitative Impact

Approach	Data Leakage	Performance Estimate	Implementation Ease	Recommended For
After Split (correct)	None	Realistic and unbiased	Moderate (use Pipelines)	All production models
Before Split (wrong)	High	Optimistic / biased upward	Easy (one transform call)	Never; quick prototyping only
No scaling (trees)	None	Realistic	Easy	Random Forest, XGBoost, Decision Trees

Table 2 – Comparison of normalization timing approaches.

5. Scikit-Learn Pipeline Best Practice

The recommended implementation uses scikit-learn `Pipeline` objects, which automatically apply `fit_transform` on training data and `transform` only on test data during cross-validation, preventing any form of leakage:^[3]

```

pipe = Pipeline([("scaler", StandardScaler()), ("clf", SVC())])
pipe.fit(X_train, y_train) → pipe.score(X_test, y_test)
  
```

This pattern also works correctly inside `GridSearchCV` and `cross_val_score`, where the scaler is re-fit on each training fold independently. ^[3]

6. Tree-Based Models: Exception

Decision trees, Random Forests, and Gradient Boosting methods split on feature thresholds that are scale-invariant: multiplying a feature by a constant does not change which threshold produces the best split. These models **do not require normalization**, removing the leakage concern entirely. ^{[3][1]}

7. Discussion and Practical Guidelines

- Always use Pipeline: guarantees leakage-free scaling inside cross-validation loops.
- Validate on a held-out set: if before-split accuracy differs markedly from after-split, leakage is likely.
- Real-world deployment: save the scaler fit on training data and apply it at inference time to new samples — never refit.
- Missing values and imputation: the same rule applies to imputers — fit on training data only.

8. Conclusion

The correct practice — normalizing **after** the train-test split, fitting the scaler exclusively on training data — is not merely a best practice but a fundamental requirement for valid evaluation. Scaling before splitting introduces statistical leakage that inflates test metrics, hides poor generalisation, and violates the core principle of ML evaluation: the test set must remain truly unseen. The scikit-learn Pipeline API enforces this correctly by design.

References

- [1] TechnicQA (2024). Should I Normalize Before or After Test-Train Split? <https://technicqa.com/should-i-normalize-before-or-after-test-train-split/>
- [2] Reddit – LearnML (2022). Should you scale your features before the train-test split? https://www.reddit.com/r/learnmachinelearning/comments/vjoo1r/should_you_scale_your_features_before_the/
- [3] Scikit-learn (2024). Preprocessing data. Official Documentation. <https://sklearn.org/stable/modules/preprocessing.html>
- [4] Stack Overflow (2018). Normalize data before or after split of training and testing data. <https://stackoverflow.com/questions/49444262/normalize-data-before-or-after-split-of-training-and-testing-data>
- [5] McCaffrey, J. (2020). Should You Normalize and Encode Data Before Train-Test Splitting or After? <https://jamesmccaffreyblog.com/2020/05/27/should-you-normalize-and-encode-data-before-train-test-splitting-or-after-splitting/>
- [6] PMC / NIH (2024). Data Preprocessing and Leakage in Clinical ML Studies. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11978981/>
- [7] ArXiv (2025). Data Leakage in Machine Learning: A Systematic Review. <https://arxiv.org/html/2506.08274v2>